

# Guía de Testing para Análisis Numérico

(Con pytest, tolerancias numéricas y buenas prácticas)

## 1. Introducción

En análisis numérico, los tests no solo verifican que el código no lance errores de sintaxis o tipo; deben **validar que los algoritmos se comportan matemáticamente bien** dentro de tolerancias razonables. Un método de Newton puede converger perfectamente en una máquina y divergir en otra por diferencias en el redondeo. Por eso, el testing numérico tiene sus propias reglas.

Esta guía te proporciona una metodología para escribir pruebas robustas, reproducibles y que documenten el comportamiento esperado de tus implementaciones.

## 2. Filosofía del testing numérico

- **Las comparaciones directas con `==` casi nunca funcionan** debido a errores de redondeo. Siempre usa `pytest.approx` o comparaciones con tolerancias absolutas/relativas.
- **Prueba la convergencia, no solo el resultado final:** comprueba que el error disminuye como se espera al refinar la discretización o aumentar las iteraciones.
- **Incluye casos patológicos** (matrices mal condicionadas, funciones con derivada nula, puntos de inicio lejanos) para verificar la robustez.
- **Los tests son documentación ejecutable:** deben ser legibles y mostrar ejemplos de uso típicos.

## 3. Herramientas básicas

| Herramienta                             | Propósito                                                        |
|-----------------------------------------|------------------------------------------------------------------|
| <code>pytest</code>                     | Framework de testing principal.                                  |
| <code>pytest.approx</code>              | Comparación numérica con tolerancias.                            |
| <code>numpy</code> y <code>scipy</code> | Proporcionan valores de referencia (oráculos).                   |
| <code>pytest-cov</code>                 | Medida de cobertura de código.                                   |
| <code>hypothesis</code> (opcional)      | Pruebas basadas en propiedades (generación automática de casos). |

Instalación en tu entorno Conda:

```
bash
conda install pytest pytest-cov numpy scipy hypothesis
```

## 4. Estructura de pruebas recomendada

Dentro de cada capítulo (ej. ch1/), mantén:

text

```
ch1/
  src/
    root_finding.py
    linear_algebra.py
  tests/
    test_root_finding.py
    test_linear_algebra.py
    conftest.py          # fixtures compartidos
    test_common.py       # utilidades comunes
  environment.yml
```

### Archivo `conftest.py` – fixtures para análisis numérico

Ejemplo de fixtures reutilizables:

python

```
import pytest
import numpy as np

@pytest.fixture
def hilbert_3x3():
    """Matriz de Hilbert 3x3 (mal condicionada)."""
    return np.array([[1, 1/2, 1/3],
                     [1/2, 1/3, 1/4],
                     [1/3, 1/4, 1/5]])

@pytest.fixture
def tolerancias():
    """Tolerancias estándar para pruebas numéricas."""
    return {"rtol": 1e-7, "atol": 1e-10}
```

## 5. Tipos de pruebas para análisis numérico

### 5.1 Prueba de convergencia (orden y error)

Verifica que tu método iterativo alcanza la solución esperada tras un número determinado de iteraciones o que el error se reduce según el orden teórico.

**Ejemplo para método de Newton** (`test_root_finding.py`):

python

```
import pytest
import numpy as np
from src.root_finding import newton

def test_newton_convergencia_cuadratica():
    f = lambda x: x**2 - 2
    df = lambda x: 2*x
    sol_exacta = np.sqrt(2)

    errores = []
    for iter in range(1, 5):
```

```

x_aprox, _ = newton(f, df, x0=1.5, tol=1e-12, max_iter=iter)
errores.append(abs(x_aprox - sol_exacta))

# El error debería reducirse cuadráticamente: error_{k+1} ~ C * error_k^2
for k in range(len(errores)-1):
    if errores[k] > 1e-8:
        ratio = errores[k+1] / (errores[k]**2)
        # La constante debe ser estable (no probamos el valor exacto)
        assert 0.1 < ratio < 2.0

```

## 5.2 Prueba de estabilidad frente a datos mal condicionados

Usa la matriz de Hilbert (muy mal condicionada) para probar un solucionador de sistemas lineales.

python

```

def test_resolucion_hilbert(hilbert_3x3, tolerancias):
    from src.linear_algebra import resolver_sistema
    b = np.array([1, 1, 1])
    x = resolver_sistema(hilbert_3x3, b)
    # Verifica que A*x ≈ b
    np.testing.assert_allclose(hilbert_3x3 @ x, b, **tolerancias)

```

## 5.3 Prueba de consistencia con librerías de referencia

Compara tu implementación con `scipy.linalg` para validar la lógica.

python

```

def test_metodo_biseccion_vs_scipy():
    from scipy.optimize import bisect as scipy_bisect
    from src.root_finding import biseccion

    f = lambda x: x**3 - x - 2
    a, b = 1.0, 2.0
    esperado = scipy_bisect(f, a, b)
    obtenido = biseccion(f, a, b, tol=1e-10)
    assert abs(obtenido - esperado) < 1e-8

```

## 5.4 Prueba de manejo de excepciones (errores esperados)

Asegura que tu código lanza las excepciones adecuadas ante entradas inválidas.

python

```

def test_newton_derivada_cero():
    f = lambda x: (x-1)**3
    df = lambda x: 3*(x-1)**2
    with pytest.raises(ValueError, match="Derivada cercana a cero"):
        newton(f, df, x0=1.0)

```

## 6. Gestión de tolerancias

**Regla de oro:** nunca compares números en coma flotante con `==`. Usa `pytest.approx` o `numpy.testing.assert_allclose`.

### Configuración recomendada

- **Tolerancia relativa por defecto:**  $1e-7$  (para la mayoría de problemas bien condicionados).
- **Tolerancia absoluta por defecto:**  $1e-10$  (para valores cercanos a cero).
- Para métodos de orden bajo o problemas mal condicionados, puedes aumentar a  $1e-5$  o  $1e-6$ .

### Uso de `pytest.approx`

python

```
def test_sum():
    assert 0.1 + 0.2 == pytest.approx(0.3, rel=1e-6, abs=1e-12)
```

### Uso de `numpy.testing.assert_allclose`

python

```
import numpy as np
from numpy.testing import assert_allclose

def test_vector():
    esperado = np.array([1.0, 2.0])
    obtenido = np.array([1.00000001, 1.99999999])
    assert_allclose(obtenido, esperado, rtol=1e-6, atol=1e-10)
```

## 7. Integración con Git y flujo de trabajo

### 7.1 Hook de pre-commit para ejecutar pruebas

Crea el archivo `.git/hooks/pre-commit` (dentro del repositorio raíz de tu proyecto multi-capítulo):

bash

```
#!/bin/bash
echo "Ejecutando pruebas numéricas antes del commit..."
pytest --cov=src --cov-fail-under=85
if [ $? -ne 0 ]; then
    echo "❌ Las pruebas fallaron o la cobertura es insuficiente. No se realiza el commit."
    exit 1
fi
echo "✅ Pruebas pasadas. Commit permitido."
```

Hazlo ejecutable: `chmod +x .git/hooks/pre-commit`.

## 7.2 Ejecución selectiva de pruebas por capítulo

Desde la raíz del proyecto:

```
bash
```

```
pytest ch1/tests/    # solo capítulo 1
pytest ch2/tests/    # solo capítulo 2
```

## 7.3 CI/CD local con act (simulando GitHub Actions)

Puedes instalar act y ejecutar localmente el mismo flujo que usarías en GitHub. Crea `.github/workflows/ci.yml`:

```
yaml
```

```
name: CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: conda-incubator/setup-miniconda@v2
      - run: conda env create -f ch1/environment.yml
      - run: conda run -n analisis_numerico pytest ch1/tests/
```

Luego ejecuta act para probar localmente.

## 8. Anexo: Errores comunes y problemas que debes evitar

### ✗ 1. Pruebas tautológicas (el error más grave)

```
python
```

```
# MAL: usa la propia función para generar el valor esperado
valor = mi_funcion(x)
assert mi_funcion(x) == valor    # ¡siempre pasa!
```

**Corrección:** calcula el esperado manualmente o con una fuente independiente.

### ✗ 2. Comparaciones directas con floats

```
python
```

```
# MAL: puede fallar por redondeo
assert newton(f, df, 1.0) == 1.4142135623730951
```

**Corrección:** usa `pytest.approx` o `assert_allclose`.

### ✗ 3. Usar tolerancias demasiado estrictas para el problema

```
python
```

```
# MAL: para un método de orden 1 con 3 iteraciones, pedir 1e-12 es irreal
assert abs(raiz - 2.0) < 1e-12
```

**Corrección:** ajusta la tolerancia al error teórico esperado (ej.  $1e-3$  para baja precisión).

## ❌ 4. No probar casos límite

Solo pruebas entradas “bonitas” (funciones suaves, matrices bien condicionadas). Olvidas:

- Funciones con asíntotas.
- Derivada nula.
- Puntos iniciales lejanos.
- Números muy grandes o muy pequeños.

## ❌ 5. Ignorar la cobertura de código

Si tus pruebas no cubren el 85% del código, hay partes que nunca se ejecutan y pueden contener errores silenciosos.

## ❌ 6. Mezclar pruebas con código de producción

No pongas `if __name__ == "__main__":` con pruebas dentro de tus módulos `src/`. Las pruebas deben vivir en `tests/`.

## ❌ 7. No documentar las tolerancias en el código

python

```
# MAL: ¿por qué 1e-7? ¿de dónde sale?  
assert error < 1e-7
```

**Corrección:** comenta o define una constante con significado:

python

```
TOLERANCIA_CONVERGENCIA = 1e-7 # basado en el error de discretización  
assert error < TOLERANCIA_CONVERGENCIA
```

## ❌ 8. Depender de valores aleatorios sin fijar la semilla

Si usas números aleatorios en tus pruebas (para generar matrices, por ejemplo), fija la semilla:

python

```
np.random.seed(42) # reproducible
```

## ❌ 9. No probar el mensaje de la excepción

python

```
# MAL: solo prueba que se lanza alguna excepción  
with pytest.raises(ValueError):  
    mi_funcion(-1)
```

```
# BIEN: verifica el mensaje para asegurar que es la correcta  
with pytest.raises(ValueError, match="digitos debe ser positivo"):  
    mi_funcion(-1)
```

## 10. Hacer pruebas demasiado lentas

No ejecutes simulaciones pesadas dentro de la suite de pruebas unitarias. Mueve esos benchmarks a otro directorio (benchmarks/). Las pruebas unitarias deben completarse en segundos.

## 9. Conclusión

Adoptar una disciplina de testing desde el principio te ahorrará horas de depuración y te dará confianza en tus algoritmos numéricos. Recuerda:

- **Prueba la convergencia, no solo el resultado final.**
- **Usa tolerancias adecuadas y documéntalas.**
- **Cubre casos patológicos y de borde.**
- **Integra las pruebas en tu flujo de Git (hooks).**
- **Mantén la cobertura por encima del 85%.**

Si interiorizas estas prácticas, tu trabajo será reproducible, profesional y estarás preparado para colaborar en equipos de investigación o desarrollo científico.

# Apéndice A: Pruebas basadas en propiedades con Hypothesis

## A.1 ¿Qué es Hypothesis?

Hypothesis es una librería que genera automáticamente casos de prueba a partir de especificaciones de tipos y propiedades. En lugar de escribir ejemplos concretos (ej. `test_con_entrada_3`), describes **qué propiedades debe cumplir tu función** y Hypothesis genera cientos de entradas aleatorias (pero reproducibles) para verificarlas.

Esto es especialmente útil en análisis numérico porque:

- Detecta errores en casos extremos que no se te habrían ocurrido.
- Prueba la estabilidad numérica frente a una amplia gama de entradas.
- Ayuda a descubrir suposiciones ocultas (ej. "mi función funciona para todo  $x$  real" pero falla en  $x=1e-200$ ).

## A.2 Instalación

bash

```
conda install hypothesis  
# o pip install hypothesis
```

## A.3 Estrategias básicas para números

| Estrategia                                                 | Propósito                                                                           |
|------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>floats()</code>                                      | Números de coma flotante (incluye <code>nan</code> , <code>inf</code> por defecto). |
| <code>floats(allow_nan=False, allow_infinity=False)</code> | Solo floats finitos.                                                                |
| <code>integers()</code>                                    | Enteros.                                                                            |
| <code>complex_numbers()</code>                             | Números complejos.                                                                  |
| <code>lists(floats())</code>                               | Listas de floats.                                                                   |
| <code>one_of(floats(), just(None))</code>                  | Unión de tipos.                                                                     |

## A.4 Ejemplo básico: propiedad de error relativo

Supón que implementaste una función que calcula la raíz cuadrada por Newton. Una propiedad fundamental es:  $\text{sqrt}(x)^2 \approx x$  para  $x \geq 0$ .

python

```
from hypothesis import given, settings  
from hypothesis.strategies import floats  
from src.root_finding import sqrt_newton  
  
@given(x=floats(min_value=0, max_value=1e6, allow_nan=False,  
allow_infinity=False))  
@settings(max_examples=200)  
def test_sqrt_propiedad_inversa(x):  
    """Para cualquier x no negativo, sqrt(x)**2 debe ser aproximadamente x."""  
    raiz = sqrt_newton(x, tol=1e-12)  
    assert raiz * raiz == pytest.approx(x, rel=1e-8, abs=1e-10)
```



Hypothesis probará con 200 valores diferentes (por defecto 100) y te mostrará el primer contraejemplo si falla.

## A.5 Prueba de estabilidad ante cancelación catastrófica

Una función que calcula  $(1 - \cos(x)) / x^2$  para  $x$  cercano a 0 sufre cancelación. Podemos probar que es aproximadamente  $1/2$  para  $x$  pequeños.

python

```
from hypothesis import given
from hypothesis.strategies import floats
import math

def funcion_estable(x):
    """Versión numéricamente estable de (1-cos(x))/x**2."""
    if abs(x) < 1e-5:
        return 0.5 - x**2/24 + x**4/720 # serie de Taylor
    return (1 - math.cos(x)) / (x**2)

@given(x=floats(min_value=-1e-3, max_value=1e-3, allow_nan=False,
allow_infinity=False))
def test_limite_cos(x):
    # Evitamos x=0 exacto para no dividir por cero
    if x == 0:
        return
    resultado = funcion_estable(x)
    # El límite cuando x->0 es 0.5
    assert resultado == pytest.approx(0.5, rel=1e-6, abs=1e-8)
```

## A.6 Generación de matrices para pruebas de álgebra lineal

Hypothesis puede generar matrices aleatorias con propiedades específicas:

python

```
from hypothesis.strategies import integers, floats, composite
import numpy as np

@composite
def matrices_cuadradas(draw, n_min=2, n_max=10, min_val=-10, max_val=10):
    """Genera una matriz cuadrada aleatoria de tamaño n x n."""
    n = draw(integers(min_value=n_min, max_value=n_max))
    elementos = draw(floats(min_value=min_val, max_value=max_val).map(lambda x:
float(x)))
    # Nota: esto no es eficiente; mejor usar hypothesis.extra.numpy
    # Pero para demostrar la idea:
    matriz = np.random.uniform(min_val, max_val, (n, n))
    return matriz

# Usando la estrategia de Hypothesis para numpy (más eficiente)
from hypothesis.extra.numpy import arrays, array_shapes

@given(arr=arrays(dtype=np.float64, shape=array_shapes(min_dims=2, max_dims=2,
min_side=2, max_side=5),
elements=floats(-10, 10)))
def test_inversa_propiedad(arr):
    # Asegurar que la matriz es invertible (condicionarla)
    if np.linalg.cond(arr) > 1e12:
        return # evitamos matrices casi singulares
    inv = np.linalg.inv(arr)
```

```
identidad = arr @ inv
np.testing.assert_allclose(identidad, np.eye(arr.shape[0]), rtol=1e-6,
atol=1e-8)
```

## A.7 Configuración de tolerancias con Hypothesis

Hypothesis permite personalizar el número de ejemplos y criterios de parada:

python

```
from hypothesis import settings, HealthCheck

settings.register_profile("ci", max_examples=500, deadline=None,
suppress_health_check=[HealthCheck.too_slow])
settings.load_profile("ci")
```

## A.8 Buenas prácticas con Hypothesis

- **No generes entradas que incluyan nan o inf** a menos que tu función los maneje explícitamente.
- **Acota los rangos** (ej. `min_value=-1e6`, `max_value=1e6`) para evitar desbordamientos.
- **Fija una semilla** si necesitas reproducibilidad total (aunque Hypothesis es determinista por defecto).
- **Combina con assume** para filtrar casos no deseados:

python

```
from hypothesis import assume
@given(x=floats())
def test_log(x):
    assume(x > 0)
    assert math.log(x) == pytest.approx(-math.log(1/x))
```

- **No abusos de assume:** si descartas muchos casos, la generación se vuelve ineficiente. Mejor restringe la estrategia directamente.

---

# Apéndice B: Fixtures complejos en pytest

## B.1 ¿Qué es un fixture complejo?

Un fixture es una función que proporciona datos o recursos reutilizables a las pruebas. Un **fixture complejo** puede implicar:

- Creación de objetos pesados (matrices, objetos simulados).
- Configuración de estado (ej. números aleatorios con semilla fija).
- Limpieza posterior (cerrar archivos, restaurar estado global).
- Dependencia entre fixtures (un fixture usa a otro).

## B.2 Fixture para problemas de análisis numérico bien condicionados

python

```
# conftest.py
import pytest
import numpy as np

@pytest.fixture
def sistema_bien_condicionado():
    """Proporciona una matriz A y vector b para un sistema lineal bien
    condicionado.
    A es diagonal dominante, cond(A) ~ 10.
    """
    A = np.array([[4, 1, 0],
                  [1, 4, 1],
                  [0, 1, 4]], dtype=float)
    b = np.array([1, 2, 3], dtype=float)
    return A, b
```

## B.3 Fixture con alcance (scope) y limpieza

Para recursos costosos (como generar una matriz grande o cargar datos), usa `scope="module"` o `"session"` y `teardown` con `yield`:

python

```
@pytest.fixture(scope="module")
def matriz_hilbert_larga():
    """Genera una matriz de Hilbert de 100x100 (costosa)."""
    import numpy as np
    n = 100
    H = np.fromfunction(lambda i, j: 1/(i+1 + j+1 -1), (n, n))
    yield H
    # Limpieza (opcional, aquí no hay recursos externos)
    print("Liberando memoria (no necesario para numpy)")
```

## B.4 Fixture que usa otros fixtures (composición)

python

```
@pytest.fixture
def sistema_mal_condicionado(sistema_bien_condicionado):
    """Toma el sistema bien condicionado y lo hace mal condicionado
    escalando una fila para casi singularidad.
    """
    A, b = sistema_bien_condicionado
    A_mal = A.copy()
    A_mal[0] = A_mal[0] * 1e-6 # casi singular
    return A_mal, b
```

## B.5 Fixture paramétrico (probar con múltiples configuraciones)

python

```
@pytest.fixture(params=[(2,1e-6), (3,1e-8), (5,1e-10)])
def configuracion_newton(request):
    """Proporciona (orden_metodo, tolerancia) para probar diferentes
    precisiones."""
    orden, tol = request.param
    return {"orden": orden, "tol": tol}
```

Uso en la prueba:

python

```
def test_newton_con_config(configuracion_newton):
    tol = configuracion_newton["tol"]
    # ... usar tol
```

## B.6 Fixture para fijar la semilla aleatoria (reproducibilidad)

python

```
@pytest.fixture(autouse=True)
def fijar_semilla():
    """Fija la semilla de numpy.random antes de cada prueba."""
    np.random.seed(42)
    yield
    # Opcional: restaurar estado anterior si fuera necesario
```

Con autouse=True se aplica a todas las pruebas automáticamente.

## B.7 Fixture que provee funciones de prueba predefinidas

python

```
@pytest.fixture
def funciones_prueba():
    """Retorna un diccionario de funciones para probar métodos de búsqueda de raíces."""
    def f1(x): return x**2 - 4
    def df1(x): return 2*x
    def f2(x): return math.exp(x) - 3*x
    def df2(x): return math.exp(x) - 3
    return {
        "polinomio": (f1, df1, 2.0, 1e-10),
        "exponencial": (f2, df2, 0.5, 1e-8),
    }

def test_newton_con_varias(funciones_prueba):
    for nombre, (f, df, sol_esperada, tol) in funciones_prueba.items():
        raiz = newton(f, df, x0=sol_esperada*0.9, tol=tol)
        assert abs(raiz - sol_esperada) < tol
```

## B.8 Fixture para capturar salida numérica (logging de errores)

python

```
@pytest.fixture
def registrar_iteraciones():
    """Registra el historial de iteraciones de un método numérico."""
    historial = []
    def callback(xk):
        historial.append(xk)
    yield callback
    # Al final, podemos analizar el historial
    print(f"Se registraron {len(historial)} iteraciones")
```

Uso en prueba:

python

```
def test_metodo_iterativo(registrar_iteraciones):
    resultado = punto_fijo(g, x0=1.0, callback=registrar_iteraciones,
max_iter=10)
    assert len(registrar_iteraciones.historial) <= 10 # Nota: necesitarías
acceder al fixture como objeto
```

**Nota:** El ejemplo anterior es simplificado. Para almacenar datos en el fixture, se puede usar un objeto mutable (ej. lista) y devolverlo.

## B.9 Buenas prácticas para fixtures complejos

1. **Nombres descriptivos:** `matriz_hilbert_10x10` en lugar de `m`.
2. **Documenta el propósito** y el rango de valores.
3. **Usa scope adecuado:** `"function"` (default) para cada prueba, `"module"` para recursos costosos que no cambian.
4. **Limpia recursos externos** (archivos, conexiones) con `yield` y luego código de limpieza.
5. **Evita efectos secundarios** entre pruebas: si un fixture modifica un estado global, restáuralo después.
6. **Combina con Hypothesis** para generar fixtures dinámicos (aunque Hypothesis tiene sus propias estrategias, puedes integrarlos).

## B.10 Ejemplo completo de un fixture complejo para un problema de EDO

python

```
@pytest.fixture(scope="module")
def problema_rigido():
    """Problema de EDO rígida:  $y' = -1000(y - \cos(t)) - \sin(t)$ ,  $y(0)=1$ .
    Solución exacta:  $y(t) = \cos(t) + 1000 \cdot \exp(-1000 \cdot t)$  (casi  $\cos(t)$ 
    rápidamente).
    """
    import numpy as np
    from scipy.integrate import odeint
    def rhs(y, t):
        return -1000*(y - np.cos(t)) - np.sin(t)
    t_span = np.linspace(0, 10, 1000)
    sol_exacta = lambda t: np.cos(t) + 1000*np.exp(-1000*t)
    return {"rhs": rhs, "t": t_span, "exacta": sol_exacta, "y0": 1.0}
```

Luego la prueba usa ese fixture para verificar diferentes integradores numéricos.